8.1 Master-Slave Devices Communication Principle

1. Introduction

This section aims to introduce you to detailed information about the master-slave relationship between the MiniArm robotic arm and different devices (such as Arduino, PC) during communication. It helps you understand how MiniArm communicates as a subordinate device with other devices and how other devices control MiniArm as a master device.

In this chapter, MiniArm is typically utilized as a subordinate device. It uses the UART serial port to transmit the information to other devices.

2. Master-Slave Relationship

In a master-slave control system, MiniArm acts as a subordinate device and other devices such as microcontrollers act as master devices.

2.1 miniArm Functions

1) Receive and parse signals sent by the master device:

   The MiniArm waits for serial signals. If data is received through the serial port, it will parse the received data based on the communication protocol to call the corresponding function.

2) Call MiniArm's functions based on received data:

   When a signal is parsed, the corresponding function of MiniArm needs to be called, such as controlling the servo movement, reading the servo angle, controlling the buzzer, and controlling the RGB lights.

3) Data encapsulation and feedback:

   When a read command is received, MiniArm needs to call the corresponding read function. Encapsulate the read data into a data packet according to the communication protocol, and send it back to the master device.

2.2 Master Devices

1) Command packing and sending:

The master device needs to package control commands and data into data packets according to the communication protocol to sends them to MiniArm.

2) Control coordination:

The master is responsible for coordinating the entire system, ensuring there are no conflicts in communication and operation between MiniArm and other devices to maintain a stable working state.

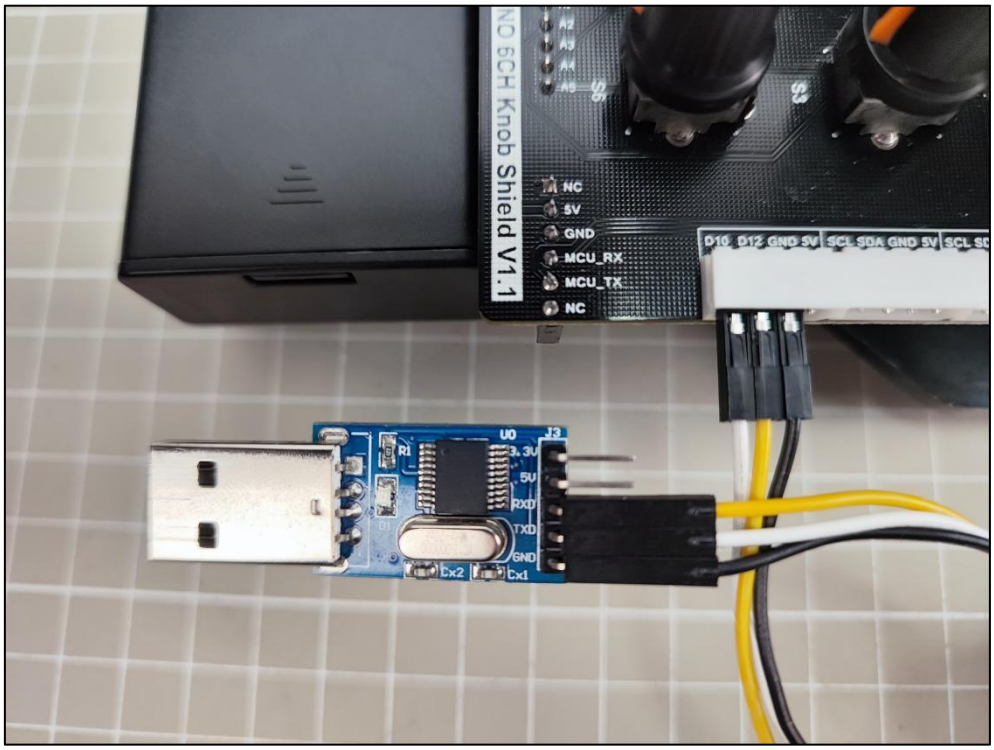
3) Data reception:

The master sends a read command to read the servo status information data sent by MiniArm. It verifies the integrity and accuracy of the received data and parses the data packet for extracting useful information.

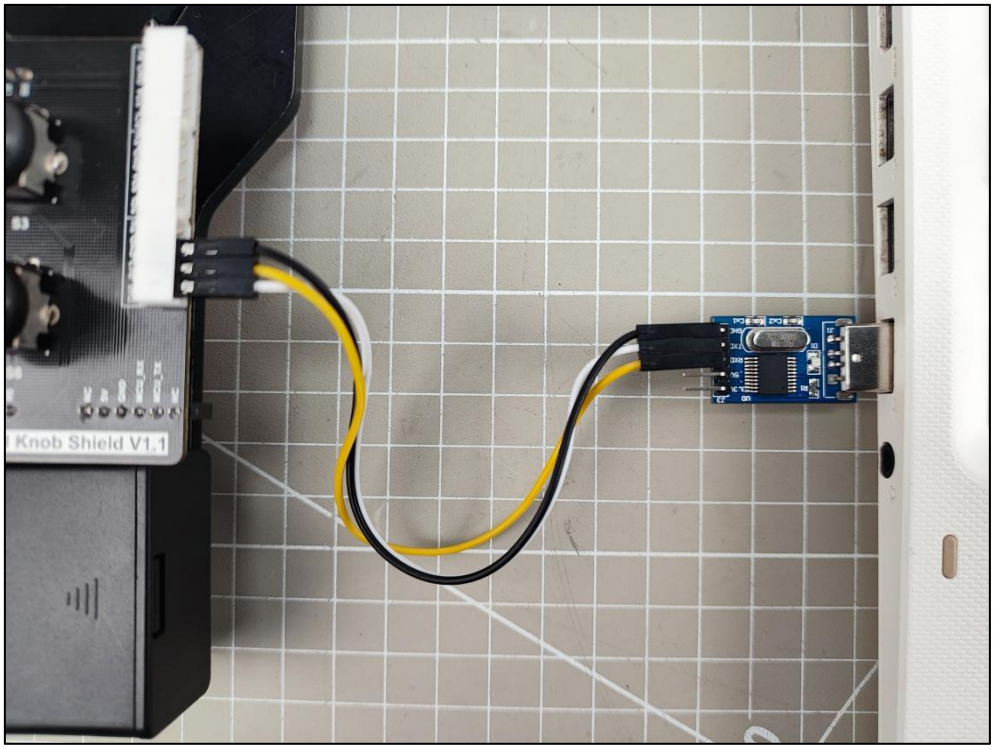
3. Hardware Connection

Take MiniArm connected to a PC as an example:

1) Use DuPont wires to respectively connect the RXD, TXD, and GND of the USB adapter to the IO12, IO10, and GND interfaces on the Arduino UNO.



2) Connect the USB adaptor to the PC.



4.Data Transmission Format

The default UART serial data transmission Format of MiniArm is as follows:

Baud rate	9600
Data bits	8
Parity	None
Stop bits	1

5. Communication Protocol

Header	Function Code	Data Length	Data Information	Parity
0xAA 0x77	func	len	data	check

Header: When the header is received, it indicates the presence of data transmission, and the header data length is fixed at 2 bytes;

Function Code: It indicates the function of an information frame;

Data Length: It indicates the amount of data that follows;

Data Information: It indicates the transmitted data information;

Parity: (Function Code + Data Length + Data) negated, and the low byte is taken.

6. Note

1) The host device and the MiniArm can have different power sources, but they must share a common ground when connected to ensure stable communication voltage levels.

2) When you wire the devices, ensure that the TX and RX pins of the UART serial port are crossed, otherwise communication will not be possible.